

TECHNOLOGY EVALUATION

Claude Code の導入経路を決める

Anthropic 直接契約と Amazon Bedrock 経由の機能差、運用上の論点、推奨する構成



この資料で答えを出すこと

Claude Code は同じ CLI だが、認証経路によって使える機能が変わる。どちらを選ぶかは要件で決まる

01 何が同じで、何が違うのか

CLI とローカルで完結する機能は全経路で同じ。差が出るのはサーバー側の機能

02 その差は業務に影響するか

欠ける機能に代替手段があるかどうかで、影響の大きさが変わる

03 どちらを選ぶか

既存の AWS 契約・データ所在・統制要件の 3 点で判断する

開発作業そのものは、経路によらず同じものが使える

同一

CLI ・ IDE 拡張 ・ サブエージェント ・ Skills ・ MCP ・ フック

CLAUDE.md メモリ、プラグイン、チェックポイント、サンドボックス、OpenTelemetry メトリクスも全経路で利用できる

1M

トークンの文脈長（Sonnet 5 等、Bedrock でも利用可）

3 社

選べるクラウド（AWS / Google / Microsoft）

IAM

Bedrock では既存の AWS 認証をそのまま使う

日々のコーディング体験を決める機能は全経路で共通。差が出るのは周辺の機能に限られる

Bedrock 経由で使えない機能と、その代替手段

機能	Anthropic 直接	Bedrock 経由	代替手段と影響
Web 検索	利用可	利用不可	URL を指定する WebFetch で代替する。調査の起点を AI に任せられない
Fast mode	利用可	利用不可	代替なし。応答の速さのみの差で、出力の品質は変わらない
Advisor	利用可	利用不可	代替なし
セッション間メッセージ・Channels	利用可	利用不可	複数セッションの連携が必要な作業は直接契約側で行う
分析ダッシュボード	利用可	利用不可	AWS 側のコスト・利用状況の仕組みで代替する（粒度は異なる）
サーバー管理設定	利用可	利用不可	managed settings ファイルの配布で統制する
クラウド実行環境	利用可	利用不可	GitHub Actions または GitLab CI/CD で代替する
定期実行（ /loop ）	間隔の自動判断可	明示指定のみ	/loop every 2 hours のように間隔を明示すれば同じ動作になる
Auto mode	全モデル	一部モデル	Sonnet 5 、 Opus 4.7 以降、 Fable 5 に限られる

欠ける機能の多くには代替がある。代替が無いのは Web 検索・Advisor・分析ダッシュボードの3つ

サブスクリプション契約でしか使えない機能

claude.ai アカウントが必要

- **ブラウザ・モバイル・Slack 上の Claude Code**
手元の端末以外から作業を始める使い方はここに含まれる
- **Routines（定期タスク）と Remote Control**
スケジュール実行と遠隔操作
- **Code Review と Ultrareview**
Code Review は Team ・ Enterprise プランのみ
- **Artifacts ・ Computer use ・ 音声入力**
プランごとに利用可否が分かれる

経路を問わず使える

- **CLI と Agent SDK**
自動化の入口はどの経路でも同じものが使える
- **サブエージェントと Skills**
社内の作法をコード化する仕組みは共通
- **フック・コマンド・プラグイン**
組織固有のワークフローを組み込める
- **MCP サーバー**
claude.ai のコネクタ連携だけはサブスクリプション限定

Bedrock 経由で欠けるものの多くは、API キー契約でも同様に使えない。境界は「Bedrock かどうか」ではなく「claude.ai アカウントかどうか」にある

Bedrock で運用する場合に先に決めておくこと

つまづきやすい点

最優先 モデルを固定していない

エイリアスは組み込みの既定値に解決される。既定は Opus のため、意図せず高い単価で動く

高 リージョンと推論プロファイル

クロスリージョン推論プロファイルの接頭辞が地域で変わる。有効化していないと 400 エラーになる

高 プロンプトキャッシュの可否

リージョンとモデルによって使えない。効かないとトークン消費が数倍になる

中 SSO の認証ループ

TLS 検査プロキシ配下で awsAuthRefresh がループする。手動ログイン運用に切り替える

先に決めること

- **ANTHROPIC_DEFAULT_OPUS_MODEL** 等でモデル ID を明示的にピン留めする
- 利用リージョンを決め、必要な推論プロファイルを有効化する
- キャッシュのトークン数が 0 でないことを配布前に実測する
- IAM ポリシーに **ListInferenceProfiles** と **GetInferenceProfile** を含める
- **Claude Code** 専用の AWS アカウントを用意し、費用と権限を分離する

モデルのピン留めを最優先で決める。未指定のまま配布すると、想定と違うモデルと単価で全社が動く

推奨する構成

この判断の根拠

- **開発作業の中核に差がない**
CLI、サブエージェント、Skills、MCP、フックはすべて共通。日々の生産性を決める部分は経路で変わらない
- **欠ける機能の大半に代替がある**
定期実行は間隔の明示、クラウド実行は CI、統制は managed settings で置き換えられる
- **代替が無いのは調査系に集中している**
Web 検索が使えない影響は、外部情報を能動的に集める業務で最も大きい
- **既存の AWS 統制をそのまま使える**
IAM、Guardrails、監査、データ所在の要件を新しい仕組みを増やさずに満たせる

推奨

統制要件が明確な組織は Bedrock を主経路とし、調査を伴う業務だけ直接契約を併用する

全社標準は Bedrock 経由とし、モデルのピン留めとリージョン設定を配布前に確定する。外部調査を日常的に行う部門には Anthropic 直接契約を追加で用意し、2 経路の併存を前提に運用ルールを作る。